

Data-Aware Tucker Alignment for Rank-Irregular Tensor Updates in Edge Federated Learning

Xinyu Liu, Chao Bu*, Zeri Zhang, Jiakai Zhou, Shuo Zhang

School of Computer Science and Engineering

Tianjin University of Technology

Tianjin, China

eieroxleon@163.com

Abstract—Federated learning on edge devices is constrained not only by limited bandwidth, but also by a structural mismatch between high-order model tensors and matrix-centric communication protocols. Convolutional kernels are naturally multi-order tensors, whereas existing compression methods often flatten them into matrices or apply data-agnostic low-rank truncation. This can disrupt tensor topology and remove locally discriminative channels under non-independent and identically distributed data. Moreover, data-aware tensor compression produces client-specific ranks and independently rotated factor spaces, making direct aggregation of compressed factors invalid. We propose Fed-PACT, a communication-efficient personalized federated learning framework for rank-irregular tensor updates on edge devices. Fed-PACT performs covariance-weighted Tucker decomposition to preserve activation-sensitive tensor modes, applies post-training INT8 quantization to reduce uplink payload, and introduces lazy decomposition to avoid redundant high-order tensor computation. On the server, Fed-PACT aligns heterogeneous tensor factors by orthogonal Procrustes analysis, pads them to the maximum rank of the current round, and applies masked aggregation to avoid averaging over artificial zeros. Experiments on a 40-class ImageNet-1k subset with Dirichlet splits show that Fed-PACT reduces total uplink payload from 3355.79 MB to 45.52 MB, achieving a $73.7\times$ communication reduction over FedAvg while maintaining 89.30 percent personalized accuracy.

Index Terms—Federated learning, edge computing, edge networking, tensor decomposition, Tucker decomposition, personalized federated learning, communication efficiency.

I. INTRODUCTION

Federated learning (FL) enables distributed edge devices to collaboratively train a model without sharing raw data [1]. However, realistic edge FL is limited by both communication bandwidth and statistical heterogeneity. Modern convolutional neural networks (CNNs), such as ResNet-50, contain tens of millions of parameters. Repeated transmission of full-precision model weights over narrowband wireless links can introduce large delay and energy consumption. At the same time, edge data are typically non-independent and identically distributed (Non-IID), which causes client drift and weakens local personalization performance [2]. This paper argues that communication-efficient edge FL also suffers from a structural mismatch. CNN convolutional kernels are naturally high-order tensors, but many compression methods treat updates as vectors or matrices. Flattening a convolutional tensor destroys the mode-wise topology among output channels, input channels, and spatial dimensions. Although low-rank matrix

compression can reduce payload, it does not explicitly preserve the tensor structure that governs convolutional feature extraction. The problem becomes more severe under heterogeneous edge data. Since each client observes a different local distribution, the activation-sensitive tensor subspaces are client-dependent. A data-aware tensor decomposition may therefore generate different truncation ranks and independently rotated factor bases across clients. The resulting compressed updates are rank-irregular tensor factors rather than shape-consistent matrices. Directly averaging such factors is mathematically invalid because the same factor column may represent different local bases on different clients. To address this edge-native tensor irregularity, we propose Fed-PACT, a personalized FL framework that compresses, aligns, and aggregates high-order tensor updates without adding trainable modules or modifying the local training pipeline. Fed-PACT uses local activation covariance to guide Tucker decomposition, thereby preserving discriminative tensor modes under Non-IID data. To aggregate heterogeneous tensor factors, the server performs orthogonal Procrustes alignment, zero-padding into a unified hyperspace, and masked averaging. A lazy update mechanism and INT8 post-training quantization further reduce computation and transmission overhead on edge devices. The contributions of this paper are summarized as follows:

- We formulate edge federated model compression as a rank-irregular tensor update problem, where non-matrixized convolutional kernels, client-specific truncation ranks, and basis mismatch jointly hinder efficient aggregation.
- We propose a covariance-weighted Tucker compression scheme that preserves locally discriminative channels while avoiding matrixization-induced topology collapse.
- We design a Procrustes-based tensor factor alignment and masked aggregation mechanism to aggregate heterogeneous Tucker factors with different ranks and rotations.
- We integrate lazy decomposition and INT8 post-training quantization to reduce both edge computation and uplink payload. Experiments show a $73.7\times$ communication reduction over FedAvg with less than two percentage points of personalized accuracy loss.

II. RELATED WORK

A. Communication-Efficient Federated Learning

FedAvg [1] is the canonical FL algorithm, and FedProx [3] improves robustness under heterogeneous clients by adding a proximal regularizer. However, both methods transmit full model parameters, which is expensive for edge networks. Quantization and sparsification reduce communication by sending low-bit values or selected coordinates [4], [5]. Recent quantized FL methods improve communication efficiency, but aggressive quantization may amplify Non-IID noise or damage important channels [6].

B. Personalized Federated Learning

Personalized FL (pFL) methods address client heterogeneity by decoupling global and local components, fine-tuning local heads, or exchanging compact representations [7]. FedRep learns a shared representation and personalized classifier [8]. FedProto communicates class prototypes instead of full model weights [9]. Fed-LoRA exchanges low-rank adapters and freezes most of the backbone [10]. These methods can reduce payload, but they do not directly aggregate the complete convolutional tensor topology in the same way as high-order tensor compression.

C. Tensor Decomposition in FL

Tensor decomposition provides a natural tool for compressing high-order model parameters. Tucker decomposition and higher-order singular value decomposition preserve multi-mode structure better than matrix flattening [11]. Recent FL studies apply tensor decomposition to power systems, medical learning, and industrial sensing [12]–[14]. However, most existing methods either use fixed ranks or do not explicitly handle the rank irregularity and basis mismatch caused by client-specific data-aware decomposition.

III. PROBLEM FORMULATION

Consider a federated system with K edge clients. Client i owns private data D_i drawn from a local distribution that may differ from other clients. For a convolutional layer, the weight parameter is a fourth-order tensor

$$W \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}, \quad (1)$$

where C_{out} and C_{in} denote output and input channels, and K_h , K_w denote spatial kernel size. The goal is to minimize local empirical risk under a communication budget:

$$\min_{\{\Theta_i\}_{i=1}^K} \sum_{i=1}^K L_i(\hat{W}_i; D_i), \quad \text{s.t.} \quad \hat{W}_i = \text{Tucker}^{-1}(\Theta_i), \quad |\Theta_i| \leq B_i, \quad (2)$$

where Θ_i is the compressed tensor payload and B_i is the edge bandwidth budget. Unlike regular matrix updates, Θ_i contains a Tucker core tensor and factor matrices:

$$\Theta_i = \{G_i, U_{i,1}, U_{i,2}, U_{i,3}, U_{i,4}\}. \quad (3)$$

Because each client has different data statistics, its truncation ranks $r_{i,n}$ can differ across both clients and tensor modes. Therefore, the server receives rank-irregular updates:

$$G_i \in \mathbb{R}^{r_{i,1} \times r_{i,2} \times r_{i,3} \times r_{i,4}}, \quad U_{i,n} \in \mathbb{R}^{I_n \times r_{i,n}}. \quad (4)$$

The aggregation challenge is to align and fuse these heterogeneous factors without destroying tensor topology or diluting valid low-rank components.

A. Structural Issues in Edge Tensor Updates

The above formulation exposes three native defects that are often hidden when model updates are treated as vectors. The first defect is *matrixization-induced topology collapse*. A convolutional kernel is not an arbitrary parameter block, but a structured mapping among output channels, input channels, and local spatial neighborhoods. Flattening W into a matrix mixes these modes and makes the compression objective depend on the chosen unfolding order. Consequently, two tensor elements that are adjacent in the spatial kernel may become far apart in the flattened representation, while elements from different channels may become artificially adjacent. This destroys the inductive bias encoded by convolution and may remove feature channels that are weak in Frobenius energy but important for local recognition. The second defect is *rank irregularity*. Under Non-IID data, different clients activate different subsets of filters. A client that mainly observes fine-grained object classes may need more channel components in deeper layers, while another client with simpler local samples may be well represented by a smaller rank. Therefore, a single global rank is either wasteful for simple clients or insufficient for complex clients. Data-aware rank selection is necessary, but it naturally produces heterogeneous tensor shapes across clients:

$$r_{i,n} \neq r_{j,n}, \quad i \neq j. \quad (5)$$

This makes the aggregation problem fundamentally different from standard FedAvg, where all clients upload tensors with identical shapes. The third defect is *basis mismatch*. Even if two clients select the same Tucker rank, their factor matrices are not guaranteed to share a common coordinate system. Singular vectors are invariant to sign flips and rotations within near-degenerate subspaces. Therefore, the d -th factor column of client i does not necessarily correspond to the d -th factor column of client j . Direct averaging can cancel meaningful components or merge unrelated local bases. Fed-PACT explicitly addresses this issue by aligning factor matrices before aggregation. These issues suggest that edge FL compression is not only a byte-reduction problem. A valid compressed update must preserve tensor topology, allow client-specific ranks, and provide a common basis for aggregation. Fed-PACT is designed around these requirements.

IV. METHOD

A. Personalized Synchronization and Payload Construction

Fed-PACT follows a decoupled personalized FL protocol. Each client keeps its classifier head and normalization statistics

local, while the shared convolutional feature extractor is synchronized and aggregated through compressed tensor updates. Let the local model of client i be divided into a personalized part W_i^p and a shared tensorized part W_i^s . At the beginning of round t , the server broadcasts $W_{global}^{s,t-1}$, and client i updates only the shared feature extractor:

$$W_i^{s,t-1} \leftarrow W_{global}^{s,t-1}, \quad W_i^{p,t-1} \leftarrow W_i^{p,t-1}. \quad (6)$$

This synchronization rule prevents the global aggregation step from overwriting client-specific decision boundaries and local feature statistics.

During local optimization, Fed-PACT can be combined with a proximal regularizer to reduce excessive client drift under Non-IID data:

$$\mathcal{L}_i = \mathcal{L}_{ce}(W_i^s, W_i^p; D_i) + \frac{\mu}{2} \|W_i^s - W_{global}^s\|_2^2. \quad (7)$$

After local training, only selected convolutional tensors in W_i^s are compressed. For layer l , the uploaded payload is

$$\mathcal{P}_i^{(l)} = \left\{ Q(G_i^{(l)}), Q(U_{i,1}^{(l)}), \dots, Q(U_{i,4}^{(l)}), \mathbf{r}_i^{(l)}, \mathbf{s}_i^{(l)}, \mathbf{z}_i^{(l)} \right\}, \quad (8)$$

where $Q(\cdot)$ denotes INT8 quantization, $\mathbf{r}_i^{(l)}$ stores the selected Tucker ranks, and $\mathbf{s}_i^{(l)}, \mathbf{z}_i^{(l)}$ store quantization scales and zero points. This payload explicitly preserves the rank-irregular tensor structure required by server-side alignment.

B. Covariance-Weighted Tucker Compression

Standard Tucker decomposition minimizes reconstruction error in weight space:

$$\min_{\hat{W}} \|W - \hat{W}\|_F^2. \quad (9)$$

However, this objective does not consider how compression error affects activations. For an input feature tensor X , the actual activation distortion can be written as

$$E_{act} = \mathbb{E}_X [\|(W - \hat{W}) * X\|_F^2]. \quad (10)$$

Let $X_{(c)}$ denote the input feature matrix unfolded along the channel dimension. The local input covariance is

$$C_i = \mathbb{E} [X_{(c)} X_{(c)}^T]. \quad (11)$$

Fed-PACT estimates C_i through a gradient-free local forward pass and updates it by exponential moving average:

$$C_i^{(t)} = (1 - \alpha_t) C_i^{(t-1)} + \alpha_t C_{batch}. \quad (12)$$

Since small local batches may produce rank-deficient covariance matrices, we add adaptive ridge regularization:

$$\tilde{C}_i = C_i + \lambda_i I, \quad \lambda_i = \epsilon \frac{\text{Tr}(C_i)}{\dim(C_i)}. \quad (13)$$

Let $R_i = \tilde{C}_i^{1/2}$. Because R_i is derived from the input-channel covariance, it is injected into the input-channel mode:

$$\tilde{W}_i = W_i \times_2 R_i. \quad (14)$$

Tucker decomposition is then performed on $\tilde{W}_i$ rather than the original weight tensor:

$$\tilde{W}_i \approx G_i \times_1 U_{i,1} \times_2 U_{i,2} \times_3 U_{i,3} \times_4 U_{i,4}. \quad (15)$$

For each mode n , Fed-PACT selects the smallest rank $r_{i,n}$ satisfying the energy criterion:

$$\frac{\sum_{j=1}^{r_{i,n}} \sigma_j^2}{\sum_{j=1}^{I_n} \sigma_j^2} \geq \eta. \quad (16)$$

To avoid feature-space collapse under aggressive compression, a rank floor is enforced:

$$r_{i,n} \leftarrow \max(2, \lfloor 0.05 I_n \rfloor). \quad (17)$$

C. INT8 Quantization and Lazy Decomposition

After Tucker decomposition, the core tensor and factor matrices are quantized using post-training INT8 affine mapping:

$$Q(x) = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z, -128, 127 \right), \quad (18)$$

where s and z are the scale and zero point. Since the data-aware Tucker step removes low-energy components before quantization, the remaining low-rank structure is more robust to low-bit representation. High-order decomposition can be expensive for edge devices. Fed-PACT therefore monitors relative weight change:

$$\Delta W_i^{(t)} = \frac{\|W_i^{(t)} - W_i^{(t-1)}\|_F}{\|W_i^{(t-1)}\|_F + \epsilon}. \quad (19)$$

If $\Delta W_i^{(t)} < \tau_{lazy}$, the client reuses the previous tensor cache instead of recomputing Tucker factors. This mechanism reduces local computation and acts as a temporal filter that avoids repeatedly uploading small Non-IID-induced fluctuations.

D. Procrustes Alignment for Rank-Irregular Factors

Because clients decompose tensors independently, their factor bases may differ by rotations, signs, or permutations. We use orthogonal Procrustes alignment [15] to correct this basis mismatch. The server first selects a reference factor $U_{ref,n}$ for each mode. Since client ranks may differ, alignment is performed on the common rank

$$h_{i,n} = \min(r_{i,n}, r_{ref,n}). \quad (20)$$

Let $U_{i,n}^h$ and $U_{ref,n}^h$ denote the first $h_{i,n}$ columns of the client and reference factors, respectively. The orthogonal rotation is obtained by

$$R_{i,n}^* = \arg \min_{R^T R = I} \|U_{ref,n}^h - U_{i,n}^h R\|_F^2. \quad (21)$$

Let

$$M_{i,n} = (U_{i,n}^h)^T U_{ref,n}^h = P \Sigma Q^T. \quad (22)$$

The closed-form Procrustes solution is

$$R_{i,n}^* = P Q^T. \quad (23)$$

The aligned factor is obtained by rotating the overlapping subspace, while columns outside the overlap are kept in their local order before padding:

$$U_{i,n}^{aligned}[:, 1 : h_{i,n}] = U_{i,n}^h R_{i,n}^*. \quad (24)$$

The corresponding core slice is rotated consistently:

$$G_i^{aligned} = G_i \times_n \bar{R}_{i,n}^T, \quad (25)$$

where $\bar{R}_{i,n}$ is an identity matrix except that its leading $h_{i,n} \times h_{i,n}$ block is replaced by $R_{i,n}^*$.

E. Masked Aggregation in a Unified Hyperspace

After alignment, clients may still have different ranks. The server computes the maximum rank for each mode:

$$R_{max,n} = \max_i r_{i,n}. \quad (26)$$

Each factor and core tensor is zero-padded to this round-wise maximum rank. To prevent padded zeros from diluting valid dimensions, Fed-PACT uses masked aggregation. For factor column d in mode n ,

$$U_{global,n}[:, d] = \frac{\sum_{i=1}^K m_{i,d} \cdot \text{Pad}(U_{i,n}^{aligned})[:, d]}{\sum_{i=1}^K m_{i,d} + \epsilon}, \quad (27)$$

where $m_{i,d} = 1$ if client i has a valid column d , and $m_{i,d} = 0$ otherwise. The core tensor is aggregated as

$$G_{global} = \frac{1}{K} \sum_{i=1}^K \text{Pad}(G_i^{aligned}). \quad (28)$$

The global tensor is reconstructed by inverse Tucker decomposition and used as a pseudo-gradient target for server momentum update.

F. Dequantization and Server Momentum Update

After receiving client payloads, the server first dequantizes each tensor component by

$$\hat{x} = s \cdot (Q(x) - z), \quad (29)$$

where s and z are transmitted with the corresponding tensor block. The aligned and aggregated Tucker components are then reconstructed into a dense convolutional tensor:

$$\hat{W}^{(t)} = G_{global}^{(t)} \times_1 U_{global,1}^{(t)} \times_2 U_{global,2}^{(t)} \times_3 U_{global,3}^{(t)} \times_4 U_{global,4}^{(t)}. \quad (30)$$

Instead of directly replacing the previous global tensor with $\hat{W}^{(t)}$, Fed-PACT treats their difference as a pseudo-gradient:

$$g^{(t)} = W_{global}^{(t-1)} - \hat{W}^{(t)}. \quad (31)$$

The server applies momentum smoothing:

$$v^{(t)} = \beta_s v^{(t-1)} + (1 - \beta_s) g^{(t)}, \quad (32)$$

and updates the global tensor by

$$W_{global}^{(t)} = W_{global}^{(t-1)} - \eta_s v^{(t)}. \quad (33)$$

This update avoids abrupt replacement by a single round of rank-irregular reconstruction and improves stability when client ranks vary across rounds.

V. THEORETICAL AND SYSTEM ANALYSIS

A. Activation-Space Preservation

The key difference between standard Tucker compression and Fed-PACT lies in the optimization space. Standard low-rank approximation minimizes the parameter reconstruction error $\|W - \hat{W}\|_F^2$. However, in neural networks, the effect of compression is measured after the weight tensor interacts with input activations. Let $E = W - \hat{W}$ denote the tensor reconstruction error. For a convolutional layer, the activation distortion can be approximated by

$$\mathbb{E}_X [\|E * X\|_F^2]. \quad (34)$$

When the input feature covariance is C_i , the distortion is no longer isotropic. Errors along frequently activated input directions are amplified, while errors along rarely activated directions have weaker influence. By injecting $R_i = \tilde{C}_i^{1/2}$ into the input-channel mode, Fed-PACT transforms the low-rank objective from uniform weight reconstruction to activation-weighted reconstruction:

$$\|(W - \hat{W}) \times_2 R_i\|_F^2. \quad (35)$$

Thus, the decomposition favors tensor components that are important for the local data distribution. This property is particularly important in personalized FL, where preserving local discriminative channels can be more valuable than minimizing global average reconstruction error.

The adaptive ridge term in (13) improves numerical stability. Since edge clients often use small local batches, the estimated covariance matrix may be low-rank or ill-conditioned. Adding $\lambda_i I$ ensures positive definiteness and bounds the condition number:

$$\kappa(\tilde{C}_i) = \frac{\lambda_{\max}(C_i) + \lambda_i}{\lambda_{\min}(C_i) + \lambda_i} \leq \frac{\lambda_{\max}(C_i) + \lambda_i}{\lambda_i}. \quad (36)$$

This prevents the square-root mapping from over-amplifying noisy covariance directions.

B. Necessity of Procrustes Alignment

The Tucker decomposition of each client is independently computed, so the factor matrices are only identifiable up to orthogonal transformations. Suppose two clients have factor matrices U_a and U_b spanning similar subspaces but represented in different coordinates. Direct averaging gives

$$\bar{U} = \frac{1}{2}(U_a + U_b). \quad (37)$$

If $U_b = U_a R$ for an orthogonal rotation R , then $\bar{U}$ may no longer be orthogonal or representative of either client subspace. In the extreme case of sign inconsistency, one component may be canceled even though both clients encode the same feature direction.

Procrustes alignment solves this coordinate inconsistency before averaging. The objective in (21) searches for the closest orthogonal rotation between a client basis and the reference basis. Since the rotation is orthogonal, it preserves inner

TABLE I
FED-PACT TRAINING AND AGGREGATION PROCEDURE

Input: clients K , communication rounds T , local epochs E , energy threshold η , lazy threshold τ_{lazy} .

1. Server initializes global model $W_{global}^{(0)}$ and broadcasts the trainable feature extractor to clients.
2. Each client synchronizes the shared extractor, keeps its personalized classifier and normalization states, and performs local training.
3. Each client estimates local activation covariance by a gradient-free forward pass and updates the covariance using exponential moving average.
4. If the relative weight change is smaller than τ_{lazy} , the client reuses its previous Tucker cache; otherwise, it performs covariance-weighted Tucker decomposition.
5. Each client applies INT8 post-training quantization to the Tucker core and factor matrices, compresses the payload, and uploads it to the server.
6. The server dequantizes all payloads, performs Procrustes alignment for factor matrices, rotates the corresponding core tensors, and pads all factors into a maximum-rank hyperspace.
7. The server performs masked aggregation for factor matrices and average aggregation for core tensors, reconstructs the global tensor, and updates the global model with server momentum.
8. The final global feature extractor is combined with client-specific heads for personalized evaluation.

products and does not change the energy of the local low-rank representation:

$$\|U_{i,n}R_{i,n}^*\|_F = \|U_{i,n}\|_F. \quad (38)$$

The corresponding inverse rotation applied to the core tensor in (25) keeps the reconstructed tensor unchanged before aggregation:

$$G_i \times_n U_{i,n} = (G_i \times_n (R_{i,n}^*)^T) \times_n (U_{i,n}R_{i,n}^*). \quad (39)$$

Therefore, alignment changes only the coordinate representation, not the local model content. This makes it suitable for server-side aggregation without requiring additional client training.

C. Masked Aggregation and Rank Validity

Zero padding is necessary for placing heterogeneous factors into a common tensor space, but naive averaging over padded dimensions introduces bias. Consider a factor column d that exists for only one client. A standard average over K clients would scale this valid component by $1/K$, even though the remaining $K - 1$ entries are artificial zeros rather than real observations. This would dilute high-rank components and discourage the global model from retaining specialized features.

The mask in (27) corrects this bias by averaging only over valid contributors. For a dimension with valid client set $\mathcal{V}_d$, the aggregation becomes

$$U_{global,n}[:,d] = \frac{1}{|\mathcal{V}_d|} \sum_{i \in \mathcal{V}_d} U_{i,n}^{aligned}[:,d]. \quad (40)$$

This preserves the magnitude of high-rank features while still allowing clients with smaller ranks to participate in lower-dimensional shared components. In this sense, masked aggregation separates two roles: low-rank dimensions capture common knowledge across many clients, while higher-rank dimensions preserve specialized knowledge from clients that require richer tensor subspaces.

D. Stability Under Rank-Irregular Aggregation

Rank-irregular aggregation introduces an additional stability challenge compared with standard FedAvg. In FedAvg, all

clients upload parameters in the same coordinate system, so averaging is performed directly in the original weight space. In Fed-PACT, each client first projects the weight tensor into a local low-rank subspace. Without alignment, the reconstructed global tensor may contain mixed bases that do not correspond to any valid client representation, leading to oscillatory updates under severe Non-IID label skew. Fed-PACT improves stability by combining alignment, masking, and momentum. Procrustes alignment reduces coordinate inconsistency before averaging, while masked aggregation prevents zero-padded dimensions from being treated as real observations. The server momentum update further smooths the reconstructed tensor trajectory across rounds. As a result, the global model does not abruptly jump to a reconstruction dominated by one round's rank configuration.

E. Communication and Computation Complexity

For a fourth-order convolutional tensor $W \in \mathbb{R}^{I_1 \times I_2 \times I_3 \times I_4}$, full-precision FL uploads

$$|\Theta_{full}| = 4 \prod_{n=1}^4 I_n \quad (41)$$

bytes under FP32 representation. After Tucker decomposition and INT8 quantization, Fed-PACT uploads approximately

$$|\Theta_{PACT}| = \prod_{n=1}^4 r_n + \sum_{n=1}^4 I_n r_n \quad (42)$$

bytes, excluding small quantization metadata. The corresponding compression ratio is approximately

$$\rho = \frac{|\Theta_{PACT}|}{|\Theta_{full}|} = \frac{\prod_{n=1}^4 r_n + \sum_{n=1}^4 I_n r_n}{4 \prod_{n=1}^4 I_n}. \quad (43)$$

When $r_n \ll I_n$, the payload is dominated by factor matrices and is much smaller than full-weight transmission.

For a multi-layer CNN, the total communication cost is the sum of the compressed payloads over all selected convolutional layers. Let $\mathcal{L}$ denote the set of layers compressed and uploaded by Fed-PACT. For layer l , the original convolutional tensor has shape $I_1^{(l)} \times I_2^{(l)} \times I_3^{(l)} \times I_4^{(l)}$, and client i selects

rank vector $\mathbf{r}_i^{(l)} = (r_{i,1}^{(l)}, r_{i,2}^{(l)}, r_{i,3}^{(l)}, r_{i,4}^{(l)})$. The total INT8 Tucker payload of client i can be approximated as

$$|\Theta_i| = \sum_{l \in \mathcal{L}} \left(\prod_{n=1}^4 r_{i,n}^{(l)} + \sum_{n=1}^4 I_n^{(l)} r_{i,n}^{(l)} \right) + |\Omega_i|, \quad (44)$$

where the first term is the core tensor size, the second term is the total factor-matrix size, and $|\Omega_i|$ denotes small metadata, including rank vectors, quantization scales, zero points, and layer identifiers. Since $|\Omega_i|$ is negligible compared with tensor factors in large convolutional layers, the payload is mainly controlled by the selected ranks.

This expression provides a natural layer-wise budget allocation mechanism. Given a client-side communication budget B_i , rank selection across layers can be viewed as

$$\min_{\{\mathbf{r}_i^{(l)}\}_{l \in \mathcal{L}}} \sum_{l \in \mathcal{L}} \mathcal{D}_i^{(l)}(\mathbf{r}_i^{(l)}) \quad \text{s.t.} \quad |\Theta_i| \leq B_i, \quad (45)$$

where $\mathcal{D}_i^{(l)}(\cdot)$ denotes the activation-space distortion introduced by truncating layer l . In practice, Fed-PACT implements this budget control through the energy threshold η and rank floor in (16)–(17). Layers whose singular energy decays rapidly automatically receive smaller ranks, while layers with slower decay retain more components.

The layer-wise form is important because CNN layers are not equally sensitive to compression. Early layers usually encode generic edge, texture, and color patterns and often have small spatial kernels. Their feature distributions are relatively shared across clients, so aggressive personalization is less necessary. Middle and deep layers contain more semantic channels and are more affected by Non-IID label skew. For these layers, preserving larger channel ranks can prevent the loss of client-specific discriminative features. Therefore, Fed-PACT can allocate communication capacity according to both tensor redundancy and data-dependent sensitivity, instead of enforcing a uniform rank ratio across the whole network.

The multi-layer payload formula also clarifies the role of lazy updates. Let $a_i^{(l,t)} \in \{0, 1\}$ indicate whether layer l of client i performs a fresh decomposition at round t . When the lazy condition is triggered, $a_i^{(l,t)} = 0$ and the cached factors are reused. If version-controlled cache transmission is enabled, the effective payload in round t can be written as

$$|\Theta_i^{(t)}| = \sum_{l \in \mathcal{L}} a_i^{(l,t)} \left(\prod_{n=1}^4 r_{i,n}^{(l,t)} + \sum_{n=1}^4 I_n^{(l)} r_{i,n}^{(l,t)} \right) + |\Omega_i^{(t)}|. \quad (46)$$

If the system still retransmits cached factors, the same expression describes the fresh-decomposition payload, while the main saving comes from computation. Thus, lazy updating always reduces decomposition cost and can also reduce communication when combined with versioned cache reuse.

The main additional client-side computation comes from covariance estimation and Tucker decomposition. Covariance estimation is performed by forward hooks without backpropagation, so it is cheaper than local training. Tucker decomposition is more expensive, but the lazy update mechanism avoids repeated decomposition when the local tensor changes

TABLE II
SINGLE-ROUND COMPRESSION AND RECONSTRUCTION ACCURACY

η	Top-1 Acc.	Top-5 Acc.	Payload	Comp. Ratio
0.90	67.78%	88.56%	10.07 MB	9.7×
0.95	75.28%	92.75%	10.42 MB	9.4×
0.97	77.01%	93.51%	10.67 MB	9.2×
0.99	77.80%	93.79%	11.07 MB	8.8×
1.00	80.00%	94.97%	15.21 MB	6.4×

only slightly. If the lazy condition is triggered for a fraction p of communication rounds, the expected decomposition cost is reduced by approximately p :

$$\mathbb{E}[C_{decomp}] = (1 - p)C_{full_decomp}. \quad (47)$$

This property is important for edge devices because the model usually changes rapidly in early rounds but enters a stable region later. Fed-PACT therefore spends more computation when structural updates are necessary and reuses cached decompositions when the tensor manifold is stable.

VI. EXPERIMENTS AND RESULTS

A. Experimental Setup

We evaluate Fed-PACT on a 40-class subset of ImageNet-1k. Data are partitioned across four clients using a Dirichlet distribution with $\alpha = 0.5$, which creates severe label skew and simulates heterogeneous edge observations. All methods use ResNet-50 pretrained on ImageNet as the backbone and run for 40 communication rounds. Each client performs one local epoch per round. Fed-PACT uses the energy threshold $\eta = 0.97$ unless otherwise specified. The lazy threshold is set to $\tau_{lazy} = 0.05$ in the multi-round experiments. The server applies orthogonal Procrustes alignment, masked aggregation, momentum-based update, and cosine annealing. We compare against FedAvg [1], FedLC [16], FedBABU [17], Fed-LoRA [10], FedProto [9], and FedRoLA [18]. The main metrics are personalized top-1 accuracy and cumulative uplink payload.

B. Single-Round Tensor Compression

Table II reports the single-round compression and reconstruction accuracy of Fed-PACT under different Tucker energy thresholds. Even when $\eta = 1.00$, Tucker-INT8 compression reduces the payload from the full ResNet-50 scale to 15.21 MB. Lowering the threshold to $\eta = 0.97$ further reduces the payload to 10.67 MB while maintaining 77.01 percent top-1 accuracy and 93.51 percent top-5 accuracy. The results show a clear trade-off between tensor rank and feature preservation. When the threshold decreases below 0.95, the payload reduction becomes marginal but accuracy drops sharply. This indicates that overly aggressive rank truncation can trigger feature-space collapse. We therefore use $\eta = 0.97$ as the default setting in the full federated experiments.

Missing figure file: pic2.png

Fig. 1. Single-round accuracy and communication payload under different Tucker energy thresholds.

TABLE III
FORTY-ROUND PERSONALIZED FL PERFORMANCE

Method	Final Acc.	Payload	Ratio	Topology
FedAvg	90.90%	3355.79 MB	1.0×	Full parameters
FedLC	89.75%	3355.79 MB	1.0×	Full parameters
FedBABU	85.35%	3344.26 MB	1.0×	Body only
Fed-LoRA	91.25%	20.79 MB	161.4×	Adapters
FedProto	92.15%	10.12 MB	331.5×	Prototypes
FedRoLA	57.30%	47.72 MB	70.3×	Random layers
Fed-PACT	89.30%	45.52 MB	73.7×	Tucker tensors

C. Personalized Federated Performance

Table III compares final personalized accuracy and total up-link payload after 40 communication rounds. FedAvg achieves 90.90 percent accuracy but requires 3355.79 MB upload per client. Fed-PACT reduces the total payload to 45.52 MB, corresponding to a 73.7× reduction, while maintaining 89.30 percent personalized accuracy. The accuracy drop is only 1.6 percentage points compared with uncompressed FedAvg. FedProto and Fed-LoRA achieve smaller payloads by exchanging prototypes or adapter parameters, but they do not directly aggregate the complete convolutional tensor topology. Fed-PACT instead compresses and aligns deep convolutional tensors, preserving the collaborative evolution of the feature extractor. FedRoLA has a similar payload scale to Fed-PACT but suffers from a large accuracy degradation, suggesting that random layer interaction may damage convergence under severe Non-IID data.

D. Effect of Lazy Tensor Updates

Fed-PACT's per-round payload decreases as training proceeds. This is caused by the lazy decomposition mechanism: once the deep feature extractor enters a stable region, small local variations are filtered by the relative-change criterion. The mechanism reduces high-order decomposition calls and suppresses redundant transmission of high-frequency local fluctuations. Under fixed wireless power and bandwidth, transmission energy is approximately proportional to payload size:

$$E_{comm} \propto |\Theta|. \quad (48)$$

Thus, replacing full-precision model transmission with INT8 Tucker factors directly reduces communication delay and edge energy consumption.

Missing figure file: pic3.png

Fig. 2. Accuracy and cumulative payload evolution during 40-round personalized federated learning.

Missing figure file: pic4.png

Fig. 3. Grad-CAM comparison among compressed personalized FL methods.

E. Visual Interpretation

Grad-CAM visualizations in Fig. 3 show that Fed-PACT preserves more concentrated activation regions on target objects under extreme compression. This suggests that covariance-weighted Tucker decomposition retains activation-sensitive channels for local data. In contrast, methods that communicate only prototypes or lightweight adapters may obtain lower payloads but can show weaker localization in cluttered scenes because the underlying convolutional tensor topology is not jointly aggregated.

F. Discussion and Limitations

The results indicate that Fed-PACT is not a replacement for prototype- or adapter-based pFL methods, but a different choice when the convolutional feature extractor needs to remain jointly trained. FedProto and Fed-LoRA communicate smaller objects, such as class prototypes or adapter weights. Fed-PACT instead communicates compressed Tucker factors of convolutional tensors, which keeps the shared backbone involved in the federated update.

The method has three main limitations. First, Tucker decomposition may introduce memory overhead on compact networks where convolutional tensors have weak low-rank structure. Second, covariance estimation requires extra forward passes, which may be unsuitable for very small edge devices. Third, the energy threshold and lazy threshold are fixed in the current implementation. Adaptive thresholds may be needed when the model changes rapidly in early rounds or when client data distributions drift over time.

VII. CONCLUSION

This paper presented Fed-PACT, a communication-efficient personalized federated learning framework for rank-irregular tensor updates on edge devices. Fed-PACT addresses three

structural bottlenecks: matrixization-induced topology collapse, client-specific rank irregularity, and factor basis mismatch. By combining covariance-weighted Tucker decomposition, INT8 quantization, lazy updates, Procrustes alignment, and masked aggregation, Fed-PACT reduces total uplink communication from 3355.79 MB to 45.52 MB on a 40-class ImageNet-1k subset, achieving a $73.7\times$ reduction over FedAvg while keeping personalized accuracy within two percentage points of the uncompressed baseline. These results show that rank-aware tensor compression and alignment can reduce edge FL communication while preserving most of the personalized accuracy.

REFERENCES

- [1] McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B. A. (2017, April). Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics* (pp. 1273-1282). Pmlr.
- [2] Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., Suresh, A. T. (2020, November). Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning* (pp. 5132-5143). PMLR.
- [3] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2, 429-450.
- [4] Alistarh, D., Grubic, D., Li, J., Tomioka, R., Vojnovic, M. (2017). QSGD: Communication-efficient SGD via gradient quantization and encoding. *Advances in neural information processing systems*, 30.
- [5] Yuzhu Mao, Zihao Zhao, Guangfeng Yan, Yang Liu, Tian Lan, Linqi Song, and Wenbo Ding. 2022. "Communication-Efficient Federated Learning with Adaptive Quantization." *ACM Trans. Intell. Syst. Technol.* 13, 4, Article 67 (August 2022), 26 pages. <https://doi.org/10.1145/3510587>
- [6] H. Chen and H. Vikalo, "Mixed-Precision Quantization for Federated Learning on Resource-Constrained Heterogeneous Devices," 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 2024, pp. 6138-6148, doi: 10.1109/CVPR52733.2024.00587.
- [7] Tan, A. Z., Yu, H., Cui, L., Yang, Q. (2022). Towards personalized federated learning. *IEEE transactions on neural networks and learning systems*, 34(12), 9587-9603.
- [8] Collins, L., Hassani, H., Mokhtari, A., Shakkottai, S. (2021, July). Exploiting shared representations for personalized federated learning. In *International conference on machine learning* (pp. 2089-2099). PMLR.
- [9] Tan, Y., Long, G., Liu, L., Zhou, T., Lu, Q., Jiang, J., Zhang, C. (2022, June). Fedproto: Federated prototype learning across heterogeneous clients. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 36, No. 8, pp. 8432-8440).
- [10] Y. Cho, A. Manoel, and G. Joshi, "Federated low-rank adaptation of large language models," *arXiv preprint arXiv:2310.03452*, 2023.
- [11] Kolda, T. G., Bader, B. W. (2009). Tensor decompositions and applications. *SIAM review*, 51(3), 455-500.
- [12] Y. Han, M. Cui, G. Luo, Q. Wang, J. Zhang and S. Bu, "A Tensor Decomposition Personalized Federated Learning Method for Distribution System State Estimation Considering Dynamic Impacts of Transmission Network," in *IEEE Transactions on Smart Grid*, vol. 17, no. 3, pp. 2555-2569, May 2026, doi: 10.1109/TSG.2026.3650953.
- [13] Chaojun Zhang, Jing Yang, Yuan Gao, Xiangli Yang, Shaojun Zou, Jieming Yang, Multi-site brain disease identification based on tensor decomposition and personalized federated learning, *Neural Networks*, <https://doi.org/10.1016/j.neunet.2025.107987>.
- [14] Wang, Y., Mu, J., Yao, Z., Wang, J., Ouyang, W., Zhou, Q., ... Chen, H. H. (2025). Knowledge Distillation and Tensor Decomposition Based Privacy-Preserving Federated Learning for Industrial IoT Radar Sensing Systems. *IEEE Internet of Things Journal*.
- [15] Gower, J. C. (1975). Generalized procrustes analysis. *Psychometrika*, 40(1), 33-51.
- [16] Zhang, J., Li, Z., Li, B., Xu, J., Wu, S., Ding, S., Wu, C. (2022, June). Federated learning with label distribution skew via logits calibration. In *International Conference on Machine Learning* (pp. 26311-26329). PMLR.
- [17] Oh, J., Kim, S., Yun, S. Y. (2021). Fedbabu: Towards enhanced representation for federated image classification. *arXiv preprint arXiv:2106.06042*.
- [18] Yan, G., Wang, H., Yuan, X., Li, J. (2024, August). FedRoLA: Robust federated learning against model poisoning via layer-based aggregation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 3667-3678).